TP6. Arquitectura del Sistema

Equipo: TAIKO (Masino, Carlos Nicolás - Becerra, Diego Fernández - Juan Carlos Gómez - Matías Alejandro)

Repositorio: https://github.com/gadsii-unlam/gadsii-taiko | **MVP en vivo:** **https://taiko-unlam.netlify.app/**

1. Diagrama de la arquitectura real del MVP

El diagrama se encuentra en el repositorio. Linkhttps://github.com/gadsii-unlam/gadsii-taiko/blob/main/docs/diseno/wireframe/UNLaM%20Wayfinder%20-%20arquitectura-mvp.svg

2. Las decisiones heredadas

Decisión	¿Quién la tomó?	¿La sostendrían hoy?	¿Qué dato del TP2 o de la prueba del TP5 la sostiene o la contradice?
Ante cualquier cambio en la base, la app vuelve a descargar todas las tablas de ocupación. Son más de 2.000 filas, en pedidos de a 1.000. Y como el panel escribe dos veces por reasignación (guarda el aula nueva y borra la original), cada cambio de aula provoca dos recargas completas. No se aplica solo la fila que cambió: un cambio de aula hace que todos los navegadores abiertos descarguen de nuevo todo el estado.	**Nadie.** Nadie la eligió. Salió de cómo se armó la carga de datos: la misma función que carga la app al abrirla se reutilizó para cada cambio en vivo.	**No, tal como está.** Alcanzó para la prueba: un solo cambio y pocos usuarios a la vez. Con muchos estudiantes conectados, cada reasignación multiplica las descargas. Habría que actualizar solo la fila que llega en el aviso de Realtime.	**Ningún dato la sostiene.** La prueba del TP5 no midió consumo de datos ni tiempos de carga: el cambio se vio en menos de un minuto, pero eso mide el aviso, no la recarga. El TP2 incluso la pone en riesgo: el contexto de uso es el celular con datos móviles o el Wi-Fi del campus, “en escenarios de apuro”. Se tomó por comodidad de construcción, no por el usuario.
El aviso de reasignación llega por Realtime solo a la app abierta y se muestra como un cartel emergente que se cierra solo a los 9 segundos. Le llega a todo el que tenga la app abierta, no solo a quien cursa esa comisión. No hay push ni service worker.	**La herramienta.** El equipo después asumió como limitación la falta de push, pero el mecanismo (aviso efímero de 9 s a todos los usuarios) vino del código generado.	**En parte.** Sí al canal en vivo y al cartel fijo “Esta clase se mudó” en la ficha. No al cierre automático a los 9 s, ni a depender de que la app esté abierta.	**La prueba la sostiene y la contradice a la vez.** **Sostiene:** 3 de 3 supieron su aula nueva en menos de 1 minuto con la app abierta. **Contradice:** U3 no vio el aviso porque bajó la vista para bajar una escalera (E10), y U2 lo tocó justo antes de que se cerrara (E07). Lo que salvó a U3 fue el cartel fijo de la ficha. El TP2 contradice además que dependa de la app abierta: el usuario pidió “notificaciones automáticas y directas”, y el caso real es el celular en el bolsillo. Que le llegue a todos y no solo a esa comisión no tiene ningún dato a favor ni en contra: la prueba tuvo un solo cambio por sesión.

3. La arquitectura de la gran visión

El diagrama se encuentra en el repositorio.

Link: https://github.com/gadsii-unlam/gadsii-taiko/blob/main/docs/diseno/wireframe/UNLaM%20Wayfinder%20-%20arquitectura-producto-completo.svg

4. Las tecnologías que agregaría la visión

Tecnología nueva	Qué le agrega a la visión	Por qué esta y no otra
Web Push con claves VAPID + service worker (protocolo)	Avisar del cambio de aula con la app cerrada, solo a los inscriptos en esa materia y comisión, que pueden desactivar el aviso.	**Contexto de uso del TP2:** el estudiante pidió "notificaciones automáticas y directas", y el uso real es el celular en el bolsillo mientras camina. El TP5 lo confirmó: U3 perdió el aviso por bajar la vista para bajar una escalera, y Realtime solo llega a la app abierta. Se elige Web Push y no una app nativa porque el producto sigue siendo una sola web que mantiene un equipo de estudiantes, sin publicar en tiendas ni mantener dos bases de código. **Costo asumido:** en iPhone, Web Push exige instalar la app en la pantalla de inicio. Por eso el alcance queda en Android.
Supabase Edge Functions (servicio, TypeScript sobre Deno)	La única pieza que corre en un servidor. Se dispara con un Database Webhook cuando se inserta una fila en cambios_aula, busca a los inscriptos en esa materia y comisión que tienen el aviso activado y envía el push con la clave privada.	**Familiaridad:** ya usamos Supabase para la base, Auth y Realtime, y esto vive en el mismo proyecto, con el mismo panel y los mismos secretos. No evaluamos otras opciones (un servidor propio en Node, Firebase Cloud Functions). **Además, hay un requisito técnico**: la clave privada VAPID no puede estar en el HTML, que es público.
Grafo de pasillos y escaleras de cada edificio, en JSON, con búsqueda de camino A* en el navegador	La ruta hasta el aula que prometía el TP1, calculada sobre los planos y no a partir del GPS solo.	**Evidencia del TP5:** la caminata dentro del edificio fue el tramo más largo de la Métrica 1. U3 buscó el aula 142 en planta baja sin saber que estaba en el 1.° piso, y U1 dudó por qué puerta entrar a Económicas. El mapa mostraba dónde estaba el aula, pero no cómo llegar. **Restricción física del campus:** con la calibración actual, el GPS tiene un error típico de 7 m y llega a 13 m en el 10 % de los puntos. No distingue un pasillo de otro, y dentro de un edificio tampoco distingue un piso de otro. Por eso la ruta se calcula sobre un grafo armado a mano con pasillos, puertas y escaleras, y el GPS sirve solo para ubicar en qué entrada del campus está el estudiante. **Por qué en el navegador y no en la base** (por ejemplo, con pgRouting): los planos ya viven en el código, y la ruta tiene que funcionar aunque Supabase no responda, igual que el mapa.

5. El componente más riesgoso

El componente con más riesgo de fallar es **el aviso de cambio en vivo**: la cadena que va de Supabase Realtime a la recarga de datos y al aviso emergente. Es el primer candidato por lo que se menciona en la consigna, porque fue lo único que estuvo cerca de fallar en el TP5. Y además es el que peor escala.

¿Qué pasó en el TP5?

Con un solo usuario a la vez:

- U3 no vio el aviso, porque bajó la vista para bajar una escalera (E10).
- U2 lo tocó justo antes de que se cerrara solo (E07).
- Lo que salvó a U3 fue el cartel fijo en la ficha del aula, no el aviso.

Es decir, la parte del MVP que sostiene la mitad de la hipótesis (enterarse de una reasignación) quedó al límite en las mejores condiciones posibles: pocos usuarios, un cambio por sesión y la app abierta.

¿Por qué empeora al crecer?

Hay cuatro cuellos que hoy no se ven:

- **Cada cambio dispara una descarga completa en todos los celulares:** Ante cualquier cambio, cada app abierta vuelve a descargar enteras las tablas de ocupación: más de 2.000 filas en tres pedidos, más uno para la de deportes. Como cada reasignación son dos escrituras (el aula nueva y el borrado de la original), son dos recargas: ocho pedidos por celular, que además se pisan entre sí porque nada las agrupa. Con 3 usuarios no se nota. Con cientos, una sola reasignación se multiplica por cientos de descargas, sobre los datos móviles que el TP2 describe como contexto de uso.
- **Hay un límite de conexiones abiertas:** Supabase permite 200 conexiones Realtime a la vez en el plan gratuito y 500 en el Pro (10.000 en el Pro sin tope de gasto). El segmento es de 30.000 a 35.000 estudiantes. Alcanza con que una fracción chica abra la app antes de la misma franja horaria para pasar el límite. En ese caso escalaríamos a un plan pago con mayor cupo. Ese límite se resuelve con presupuesto y no obliga a cambiar la arquitectura, pero deja el costo del aviso atado a la cantidad de usuarios conectados.
- **El tipo de suscripción que usa el MVP no está pensado para mucha gente:** Hoy cada celular queda suscripto directamente a la tabla de ocupación de la base de datos. Cada vez que esa tabla cambia, el servidor revisa, uno por uno, si cada suscriptor tiene permiso para ver el cambio, y recién entonces se lo envía. Ese trabajo se hace en orden, en un solo hilo, así que crece con la cantidad de cambios multiplicada por la cantidad de celulares conectados. La propia documentación de Supabase recomienda otro mecanismo para escalar (Broadcast): el cambio se publica una sola vez y se reparte a todos, sin revisarlo para cada uno. Con pocos usuarios, como en la prueba, la diferencia no se nota.

Con el segmento completo, el aviso puede llegar tarde justo cuando más estudiantes están mirando la app.

- **Si se corta la conexión, la app no se pone al día:** El canal se suscribe sin controlar el estado de la conexión. En Android, si se apaga la pantalla o el celular pasa del Wi-Fi del campus a los datos móviles, los cambios que se publiquen en ese rato se pierden: al volver, el mapa sigue mostrando el estado viejo y el aviso nunca aparece. No se vio en la prueba del TP5, porque la app estuvo abierta y en primer plano; surge de revisar el código, y es otro motivo para el push.

¿Por qué este y no otro componente?

El mapa, los planos y el buscador trabajan con datos que ya están en el código o en memoria, y Netlify sirve un archivo estático. Esa parte crece sin cambios. El aviso, en cambio, combina todo lo frágil:

- Le llega a todos y no solo a los afectados.
- Depende de que la app esté abierta.
- Recarga todo ante cada cambio.
- Usa el mecanismo de Realtime que la propia documentación desaconseja a escala.
- No se pone al día si se corta la conexión.

Y es justo el componente que el TP2 reveló como necesidad: **“notificaciones automáticas y directas”**.